

Redes Adversárias Generativas — parte 1

Inteligência Artificial Generativa

Artigo seminal

Goodfellow *et al.*, 2014

Generative Adversarial Nets

Ian J. Goodfellow*, Jean Pouget-Abadie†, Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair‡, Aaron Courville, Yoshua Bengio§
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

I. Goodfellow et al., "Generative Adversarial Nets," *Advances in Neural Information Processing Systems*, vol. 27, 2014.

Artigo seminal

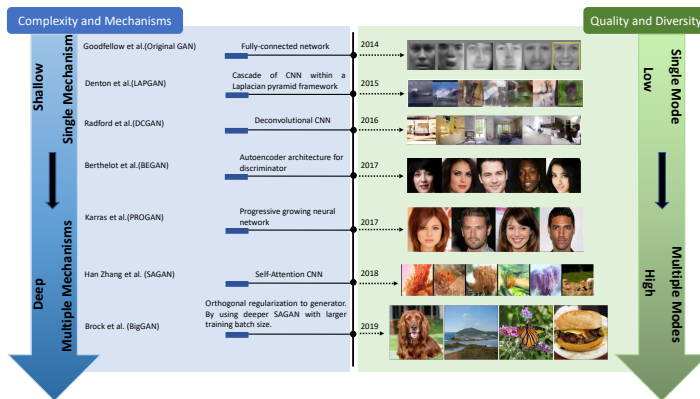
Goodfellow *et al.*, 2014



I. Goodfellow et al., "Generative Adversarial Nets," *Advances in Neural Information Processing Systems*, vol. 27, pp. 3063–3071, 2014.

Linha do tempo das GANs

Evolução das arquiteturas



Z. Wang et al., "Generative Adversarial Networks in Computer Vision: A Survey and Taxonomy," *ACM Comput. Surv.*, vol. 54, no. 2, Art. no. 37, Mar. 2022.

“Esta pessoa não existe”

Demonstração

Imagens de faces geradas usando modelo StyleGAN2 (Karras et al., 2020):

<https://thispersondoesnotexist.com/>

Redes Adversárias Generativas

Definição

- Redes Adversárias Generativas [*Generative Adversarial Networks*, GANs] são modelos **não supervisionados** para geração de dados sintéticos.
 - Não supervisionados no sentido de que **não exigem rótulos** associados ao conjunto de treinamento.
- **Objetivo:** gerar amostras plausíveis, indistinguíveis das amostras reais.
- Não modelam explicitamente a distribuição de probabilidade dos dados.
 - Não permitem avaliar diretamente a verossimilhança de um ponto sob a distribuição aprendida.

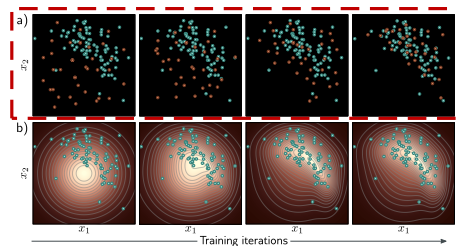
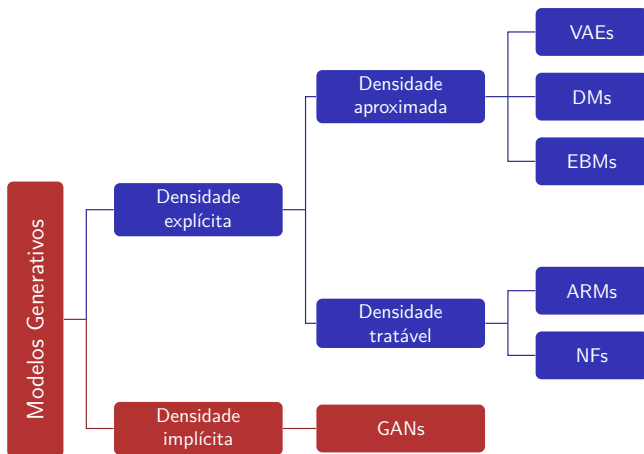
Redes Adversárias Generativas

Aplicações

- GANs podem ser utilizadas para a geração de diversos tipos de dados, por exemplo:
 - áudio
 - texto
 - vídeo
 - grafos
 - imagens
- Obtiveram grande sucesso na geração de imagens.

Taxonomia dos modelos generativos

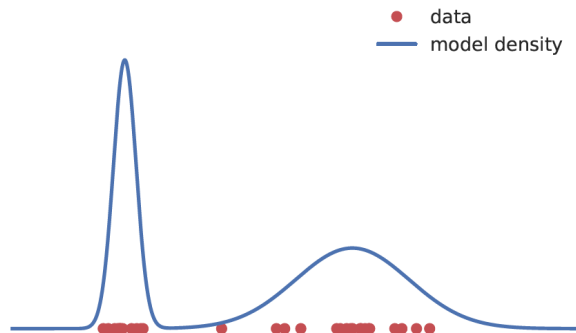
Em relação à modelagem de $p(x)$



S. J. D. Prince. *Understanding Deep Learning*, The MIT Press, 2023.

Densidade implícita vs. explícita

Tipos de modelos



(a) *Prescribed generative model.*

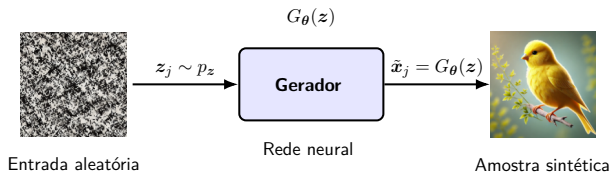


(b) *Implicit generative model.*

Arquitetura das GANs

O gerador

- **Objetivo:** gerar novas amostras $\{\tilde{x}_j\}$ extraídas da mesma distribuição que um conjunto de dados de treinamento reais $\{x_i\}$.
- Passos para a geração:
 - 1 Amostramos $z_j \sim p_z$, tipicamente $p_z = \mathcal{N}(\mathbf{0}, \mathbf{I})$.
 - 2 Utilizamos z_j como entrada para uma rede neural com parâmetros θ (gerador) gerar uma amostra sintética $\tilde{x}_j$:



- O treinamento visa ajustar os parâmetros do gerador para que $\{\tilde{x}_j\}$ pareça se originar do conjunto de dados real $\{x_i\}$.

Arquitetura das GANs

O gerador

Como definir a similaridade entre $\{\tilde{x}_j\}$ e $\{x_i\}$?

Arquitetura das GANs

O discriminador

Como definir a similaridade entre $\{\tilde{x}_j\}$ e $\{x_i\}$?

- GANs utilizam o princípio de que as amostras sintéticas $\{\tilde{x}_j\}$ devem ser estatisticamente indistinguíveis das amostras reais.
- Utilizam uma **segunda rede neural** com parâmetros ϕ (**discriminador**) para classificar suas entradas como reais ou sintéticas (**classificação binária**).
 - Se não for possível distinguir entre as amostras reais e sintéticas, então alcançamos o objetivo.
 - Caso contrário, o discriminador fornece um sinal que pode ser utilizado para treinar o gerador.

Obs.: Os rótulos (real/sintético) são **gerados automaticamente**, e não anotados manualmente.

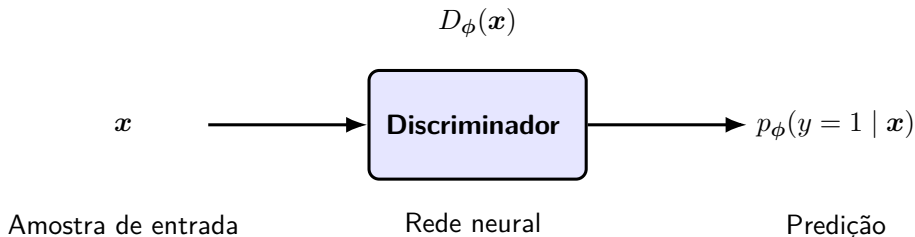
Arquitetura das GANs

O discriminador

Saída do discriminador

A saída do discriminador é a probabilidade da sua entrada x ser uma amostra real, $p(\text{real} \mid \text{entrada})$:

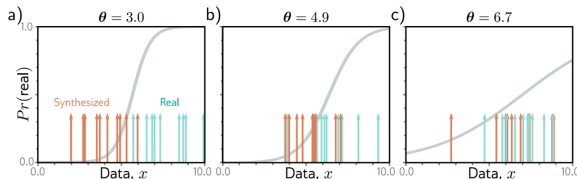
$$p_{\phi}(y = 1 \mid x) = D_{\phi}(x), \quad p_{\phi}(y = 0 \mid x) = 1 - D_{\phi}(x)$$



Exemplo unidimensional

Gerador com um único parâmetro

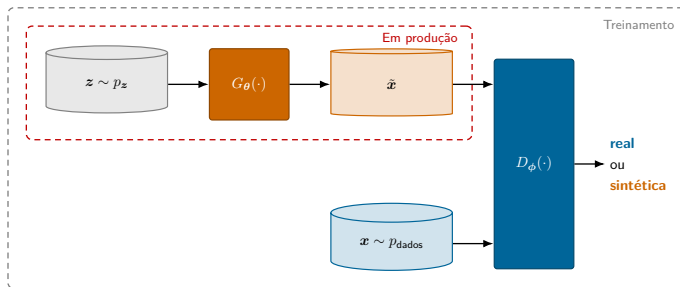
- Espaço de dados unidimensional com $z_j \sim \mathcal{N}(0, 1)$.
- Gerador simples (parametrizado por θ) da forma $\tilde{x}_j = G_\theta(z_j) = z_j + \theta$.
- Discriminador (parametrizado por ϕ) aprende uma fronteira de decisão.
 - A sigmoide representa a probabilidade da entrada do discriminador ser uma amostra real.
- O parâmetro θ é ajustado para que $\tilde{x}_j$ coincida com x_i , tornando-os indistinguíveis.
- O gerador e o discriminador são treinados alternadamente.



Exemplo unidimensional

Gerador com um único parâmetro

- Ao final do processo, idealmente o discriminador não consegue mais distinguir as amostras reais das sintéticas; ele passa a operar ao nível do acaso ($\approx 50\%$ de acurácia em um conjunto balanceado) e é **descartado**.



Adaptado de B. Lique et al., *Mathematical Engineering of Deep Learning*, CRC Press, 2024.

Arquitetura das GANs

Resumo

A arquitetura das GANs é composta por dois componentes:

1 Gerador (G)

- Mapeia variáveis latentes $z \sim p_z$ para o espaço dos dados:

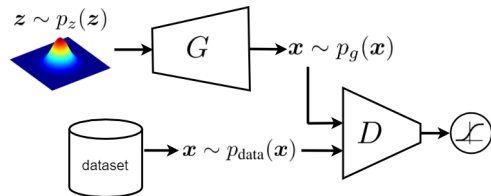
$$\tilde{x} = G_{\theta}(z).$$

2 Discriminador (D)

- Estima a probabilidade

$$D_{\phi}(x) \in [0, 1]$$

de que x seja uma amostra real.



B. Ghogoh et al., "Generative adversarial networks and adversarial autoencoders: Tutorial and survey," *arXiv preprint arXiv:2111.13282*, 2021. [Online]. Available: <https://arxiv.org/abs/2111.13282>.

Função de perda do discriminador

Distribuições e rótulos

Temos duas distribuições:

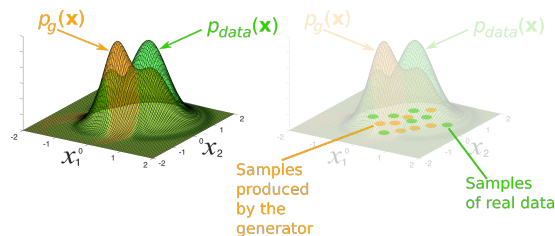
$$p_{\text{dados}}(\mathbf{x}) \quad \text{e} \quad p_g(\mathbf{x}).$$

- $p_{\text{dados}}(\mathbf{x})$: distribuição dos dados reais.
- $p_g(\mathbf{x})$: distribuição induzida pelo gerador.

- Introduzimos a variável aleatória binária y (rótulo):

- $y = 1$: dados reais.
- $y = 0$: dados sintéticos.

$$\mathbf{x} \mid y = 1 \sim p_{\text{dados}}, \quad \mathbf{x} \mid y = 0 \sim p_g.$$

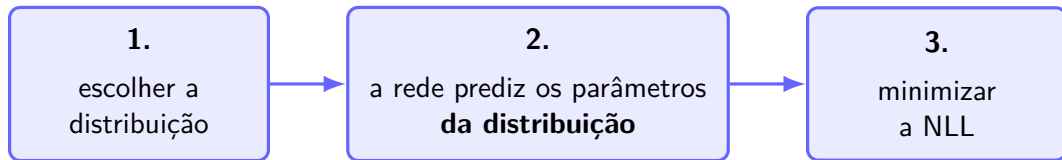


A. Creswell et al., "Generative Adversarial Networks: An Overview," in IEEE Signal Processing Magazine, vol. 35, no. 1, pp. 53-65, Jan. 2018.

Funções de perda

Origem

O discriminador resolve um problema de **classificação binária**. Para construir a função de perda e treinar o modelo seguimos a receita:



Função de perda do discriminador

Perda de uma amostra

Passo 1. Escolher a distribuição. Como $y \in \{0, 1\}$: **Bernoulli** de parâmetro λ .

$$y \mid \mathbf{x} \sim \text{Ber}(\lambda) \quad \implies \quad p(y \mid \lambda) = \lambda^y (1 - \lambda)^{1-y}$$

Passo 2. A rede, de parâmetros ϕ , prediz o parâmetro λ **da distribuição**. Como $\lambda \in [0, 1]$, a saída passa por uma sigmoide:

$$\lambda = D_\phi(\mathbf{x}) = \text{sig}[f_\phi(\mathbf{x})] \quad \implies \quad p_\phi(y \mid \mathbf{x}) = D_\phi(\mathbf{x})^y [1 - D_\phi(\mathbf{x})]^{1-y}$$

Passo 3. Minimizar a log-verossimilhança negativa. Para uma única observação:

$$\ell(y, D_\phi(\mathbf{x})) = -\log p_\phi(y \mid \mathbf{x})$$

▷ Passo 3: NLL

$$= -\log [D_\phi(\mathbf{x})^y (1 - D_\phi(\mathbf{x}))^{1-y}]$$

▷ Passos 1 e 2

$$= \boxed{-y \log D_\phi(\mathbf{x}) - (1 - y) \log [1 - D_\phi(\mathbf{x})]}$$

▷ propriedades do logaritmo
entropia cruzada binária

Função de perda do discriminador

Perda do lote

Somando a perda pontual sobre as n amostras de um lote e tomando a média:

$$-\frac{1}{n} \sum_{i=1}^n \left[y_i \log D_{\phi}(\mathbf{x}_i) + (1 - y_i) \log(1 - D_{\phi}(\mathbf{x}_i)) \right]$$

Como $y_i \in \{0, 1\}$, cada amostra contribui com **apenas um** dos dois termos:

$$\underbrace{-\frac{1}{n} \sum_{\text{reais}} \log D_{\phi}(\mathbf{x}_i)}_{y_i=1} \quad \underbrace{-\frac{1}{n} \sum_{\text{ sintéticas}} \log(1 - D_{\phi}(\tilde{\mathbf{x}}_j))}_{y_j=0}$$

Com o lote **balanceado** (m reais e m sintéticas, $n = 2m$) e $\tilde{\mathbf{x}}_j = G_{\theta}(\mathbf{z}_j)$:

$$-\frac{1}{2m} \sum_{i=1}^m \log D_{\phi}(\mathbf{x}_i) - \frac{1}{2m} \sum_{j=1}^m \log(1 - D_{\phi}(G_{\theta}(\mathbf{z}_j)))$$

Função de perda do discriminador

Versão em esperanças

Perda do discriminador

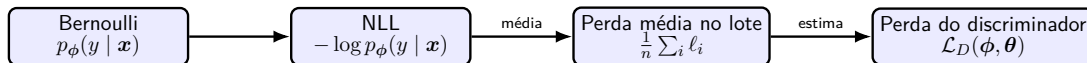
As médias amostrais **estimam** as esperanças correspondentes. Escrevemos então a perda do discriminador como:

$$\mathcal{L}_D(\phi, \theta) = \frac{1}{2} \mathbb{E}_{x \sim p_{\text{dados}}} [-\log D_{\phi}(x)] + \frac{1}{2} \mathbb{E}_{z \sim p_z} [-\log (1 - D_{\phi}(G_{\theta}(z)))]$$

- O fator $\frac{1}{2}$ vem do lote balanceado: metade das amostras é real, metade é sintética.
- A segunda esperança é tomada sobre $z \sim p_z$, e não sobre $x \sim p_g$: **não precisamos conhecer** $p_g(x)$, basta amostrar z e passar pelo gerador.

Função de perda do discriminador

Resumo



Função de perda do discriminador

Resumo

	Classificação binária tradicional	Discriminador da GAN
Entrada	x (classes positiva e negativa)	x (real ou sintética)
Saída	$p(\text{positiva} \mid \text{entrada})$	$p(\text{real} \mid \text{entrada})$
Rótulo	$y \in \{0, 1\}$	
Passo 1: distribuição	Bernoulli(λ)	
Passo 2: a rede prediz	$\lambda \in [0, 1]$, utilizando a sigmoide na saída da rede	
Passo 3: perda	entropia cruzada binária	
Lote	$\{(x_i, y_i)\}_{i=1}^{2m}$	$\underbrace{\{(x_i, 1)\}_{i=1}^m}_{\text{reais, } x_i \sim p_{\text{dados}}} \cup \underbrace{\{(\tilde{x}_j, 0)\}_{j=1}^m}_{\text{sintéticas, } \tilde{x}_j = G_{\theta}(z_j) \sim p_g}$
Distribuição	p_{dados} estacionária	p_{dados} estacionária, p_g não estacionária

Função de perda do gerador

Jogo de soma zero e função de valor

- O discriminador e o gerador competem em um **jogo de soma zero** [*zero-sum game*] $\rightarrow$ a soma dos custos de todos os jogadores é zero:

$$\mathcal{L}_G(\phi, \theta) + \mathcal{L}_D(\phi, \theta) = 0 \quad \Rightarrow \quad \boxed{\mathcal{L}_G(\phi, \theta) = -\mathcal{L}_D(\phi, \theta)} \quad \triangleright \text{perda do gerador}$$

- Podemos resumir o jogo com uma **função de valor** V :

$$\begin{aligned} V(\phi, \theta) &= -\mathcal{L}_D(\phi, \theta) \\ &= \boxed{\frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{dados}}} [\log D_{\phi}(\mathbf{x})] + \frac{1}{2} \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}} [\log (1 - D_{\phi}(G_{\theta}(\mathbf{z})))]} \end{aligned}$$

Jogo minimax

Formulação

Podemos escrever o problema de aprendizado como um jogo **minimax**, pois sua solução envolve:

- **Maximização** em um laço interno.
 - Para um gerador fixo, encontrar o melhor discriminador.
- **Minimização** em um laço externo;
 - Encontrar um gerador que engane esse discriminador.

$$\theta^* = \arg \min_{\theta} \left[\max_{\phi} V(\phi, \theta) \right]$$

Jogo minimax

Passo do discriminador e do gerador

Discriminador

Dado um gerador G_θ **fixo**, o discriminador resolve um problema de classificação binária entre amostras reais $\mathbf{x} \sim p_{\text{dados}}$ e sintéticas $\tilde{\mathbf{x}} = G_\theta(\mathbf{z})$, $\mathbf{z} \sim p_z$, maximizando a função de valor, ou, equivalentemente, minimizando a entropia cruzada. Seja D_ϕ^* o discriminador ótimo, então seus parâmetros são dados por:

$$\phi^* = \arg \max_{\phi} V(\phi, \theta)$$

Gerador

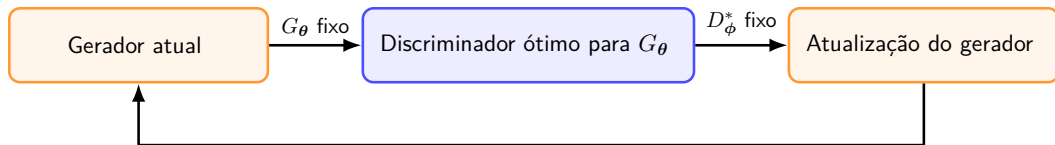
Dado o discriminador ótimo D_ϕ^* anterior, o gerador procura resolver o seguinte problema:

$$\arg \min_{\theta} V(\phi^*, \theta)$$

Jogo minimax

Alternância entre as redes

- Quando o gerador muda para melhor enganar o discriminador, a distribuição sintética muda.
- Portanto, o discriminador ótimo para o gerador anterior deixa de ser ótimo para o novo gerador e é necessário resolver novamente $\arg \max_{\phi} V(\phi, \theta)$.



Treinamento

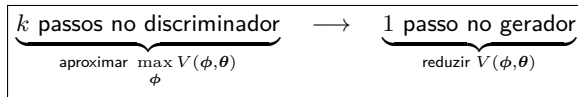
Alternância entre as redes

Para treinar uma GAN precisamos alternar entre o ajuste dos parâmetros do gerador e do discriminador.

- O discriminador é atualizado para aproximar:

$$\hat{\phi} \approx \phi^* = \arg \max_{\phi} V(\phi, \theta).$$

- Em seguida, o gerador é atualizado mantendo ϕ fixo.



- O hiperparâmetro k controla quantos passos do discriminador são feitos para cada passo do gerador ($k \uparrow \Rightarrow$ melhor a aproximação para D_{ϕ}^*).

Na prática, $k = 1$ geralmente já é suficiente.

Treinamento

Os dois problemas de otimização

Com dados, a função de valor é aproximada por:

$$V(\phi, \theta) \approx \hat{V}(\phi, \theta) = \frac{1}{2m} \sum_{i=1}^m \log D_{\phi}(\mathbf{x}_i) + \frac{1}{2m} \sum_{j=1}^m \log [1 - D_{\phi}(G_{\theta}(\mathbf{z}_j))].$$

Desprezando a constante positiva $1/2$ no processo de otimização, temos:

- Discriminador

$$\hat{\phi} = \arg \max_{\phi} \left[\frac{1}{m} \sum_{i=1}^m \log D_{\phi}(\mathbf{x}_i) + \frac{1}{m} \sum_{j=1}^m \log [1 - D_{\phi}(G_{\theta}(\mathbf{z}_j))] \right]$$

- Gerador

$$\hat{\theta} = \arg \min_{\theta} \left[\max_{\phi} \left[\frac{1}{m} \sum_{i=1}^m \log D_{\phi}(\mathbf{x}_i) + \frac{1}{m} \sum_{j=1}^m \log [1 - D_{\phi}(G_{\theta}(\mathbf{z}_j))] \right] \right]$$

Treinamento

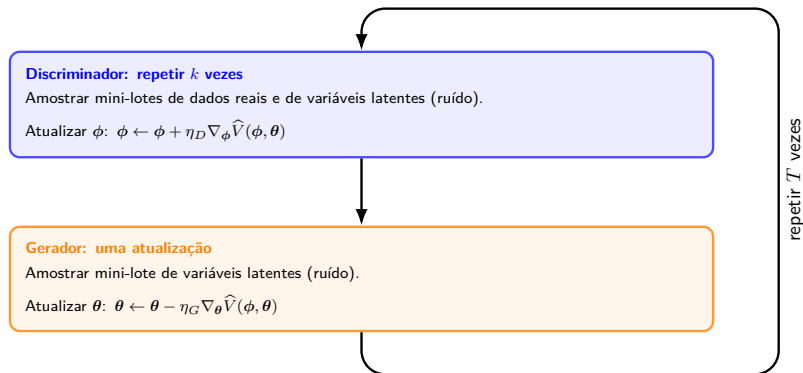
Passos de cada iteração

- Em cada passo:
 - Geramos um lote de variáveis latentes z_j utilizando a distribuição base;
 - Alimentamos o gerador com o lote de z_j e criamos as amostras sintéticas $\tilde{x}_j = G_\theta(z_j)$;
 - Escolhemos um lote de amostras reais de treinamento x_i ;
 - Com os dois lotes, aplicamos um algoritmo de otimização em cada função de perda.

Treinamento

Laço de treinamento

Ao treinar uma GAN, obtemos uma sequência de parâmetros das redes: $\underbrace{(\phi^{(1)}, \theta^{(1)})}_{\text{iteração 1}}, \underbrace{(\phi^{(2)}, \theta^{(2)})}_{\text{iteração 2}}, \dots, \underbrace{(\phi^{(T)}, \theta^{(T)})}_{\text{iteração } T}$.



Treinamento

Algoritmo

Entradas: T : número de iterações; m : tamanho do mini-lote; k : número de atualizações do discriminador;
 η_D e η_G : taxas de aprendizado do discriminador e do gerador, respectivamente.

```
1 for  $t = 1, \dots, T$  do
2   for  $s = 1, \dots, k$  do                                     ▷ Atualização do discriminador
3     Amostrar mini-lote com  $m$  amostras dos dados reais:  $\{x_1, \dots, x_m\} \sim p_{\text{dados}}$ 
4     Amostrar mini-lote com  $m$  amostras de ruído:  $\{z_1, \dots, z_m\} \sim p_z$ 
5     Atualizar o discriminador por ascensão estocástica do gradiente (mantendo  $\theta$  fixo):
```

$$\phi \leftarrow \phi + \eta_D \nabla_{\phi} \left\{ \frac{1}{m} \sum_{i=1}^m \left[\log D_{\phi}(x_i) + \log(1 - D_{\phi}(G_{\theta}(z_i))) \right] \right\}$$

```
6   end for
7   Amostrar mini-lote com  $m$  amostras de ruído:  $\{z_1, \dots, z_m\} \sim p_z$                                      ▷ Atualização do gerador
8   Atualizar o gerador por descida estocástica do gradiente (mantendo  $\phi$  fixo):
```

$$\theta \leftarrow \theta - \eta_G \nabla_{\theta} \left\{ \frac{1}{m} \sum_{j=1}^m \log(1 - D_{\phi}(G_{\theta}(z_j))) \right\}$$

```
9 end for
```

Adaptado do algoritmo original (Goodfellow *et al.*, 2014), de (M. Megahed and A. Mohammed, 2024) e de (C. M. Bishop and H. Bishop, 2024).

Treinamento

O jogo reescrito como duas perdas

Na prática, tipicamente os dois problemas de otimização são resolvidos como minimização de **duas perdas**. Maximizar $\hat{V}$ é minimizar $-\hat{V}$, e constantes positivas não mudam o ponto ótimo. Para a GAN minimax original:

Perda do discriminador: **minimiza** a entropia cruzada (rótulos **verdadeiros**: 1 nas reais, 0 nas sintéticas)

$$\hat{\mathcal{L}}_D = -\frac{1}{m} \left[\sum_{i=1}^m \log D_{\phi}(\mathbf{x}_i) + \sum_{j=1}^m \log (1 - D_{\phi}(G_{\theta}(\mathbf{z}_j))) \right]$$

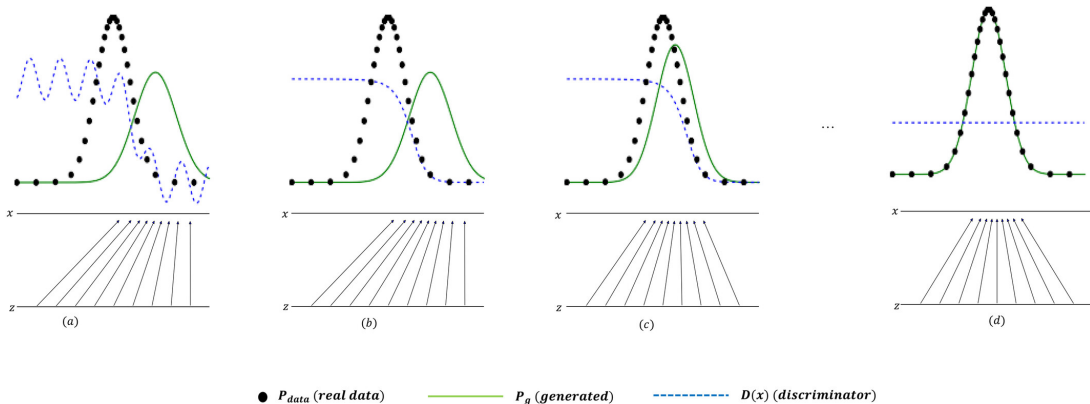
Perda do gerador (**minimax**): **maximiza** a entropia cruzada (rótulos **verdadeiros**: 0 nas sintéticas)

$$\hat{\mathcal{L}}_G^{\text{MM}} = \frac{1}{m} \sum_{j=1}^m \log (1 - D_{\phi}(G_{\theta}(\mathbf{z}_j)))$$

Obs.: Minimizar $\hat{\mathcal{L}}_D$ é a ascensão do slide anterior (já sem o fator 1/2), só com o sinal trocado; a atualização do gerador já era uma minimização. Note que $\hat{\mathcal{L}}_G^{\text{MM}} = -\hat{\mathcal{L}}_D^{(\text{sint.})}$ (parcela dos dados sintéticos em $\hat{\mathcal{L}}_D$): as duas redes atuam sobre o **mesmo termo** em direções opostas. Em relação à identidade $\mathcal{L}_G = -\mathcal{L}_D$ do slide *Jogo de soma zero e função de valor*, as esperanças são substituídas por médias amostrais e o objetivo do gerador sofre duas modificações que preservam o $\arg \min_{\theta}$: a supressão do termo $\frac{1}{m} \sum_i \log D_{\phi}(\mathbf{x}_i)$, **constante em θ** e portanto de gradiente nulo para o gerador, e o descarte do fator 1/2, constante multiplicativa positiva.

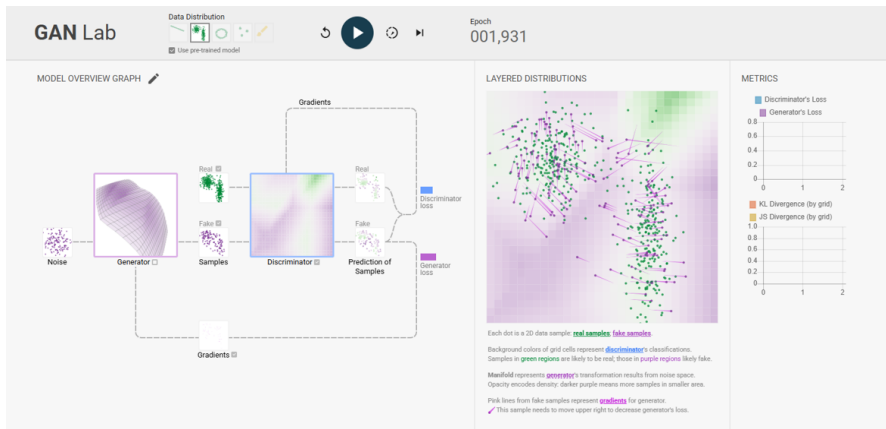
Treinamento

Processo de treinamento de uma GAN



M. Cobbinah et al., "Diversity in stable GANs: A systematic review of mode collapse mitigation strategies," Engineering Reports, vol. 7, no. 6, Art. no. e70209, 2025.

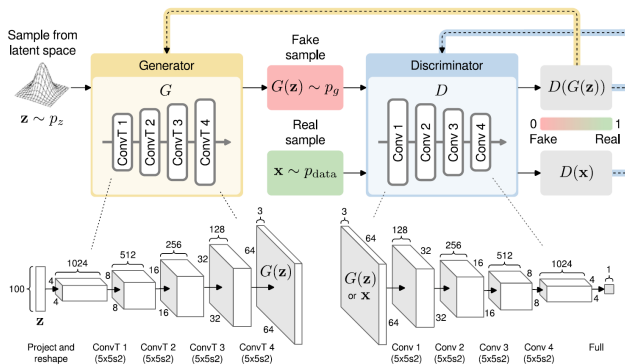
GAN Lab



<https://poloclub.github.io/ganlab/>

Deep convolutional GAN [DCGAN]

Deep convolutional GAN (DCGAN): arquitetura especializada para geração de imagens.



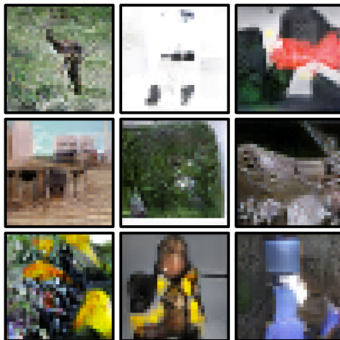
V. L. Trevisan de Souza et al., "A review on generative adversarial networks for image generation," *Computers & Graphics*, vol. 114, pp. 13–25, 2023.

Deep convolutional GAN [DCGAN]

a)



b)



c)



S. J. D. Prince. *Understanding Deep Learning*, The MIT Press, 2023.

Dificuldades no treinamento

Algumas dificuldades no treinamento que podem levar a falhas nas GANs são:

- Sensibilidade aos hiperparâmetros
- Função de perda do gerador não informativa
- *Mode dropping* e *mode collapse*
- Desvanecimento dos gradientes [*vanishing gradients*]

Dificuldades no treinamento

Sensibilidade a hiperparâmetros

Diversos fatores afetam o treinamento complexo das GANs. O modelo é altamente sensível a pequenas variações, de forma que o seu sucesso depende de uma sintonização manual meticulosa de muitos hiperparâmetros, entre eles:

- dimensionalidade do espaço latente;
- arquiteturas do gerador e discriminador;
- taxa de aprendizado;
- funções de ativação;
- tamanho dos mini-lotes de treinamento;
- entre outros.

Dificuldades no treinamento

Sensibilidade a hiperparâmetros

Por exemplo, para treinar uma DCGAN, foram necessárias escolhas de arquitetura e hiperparâmetros muito específicos:

- uso de convoluções com *stride* para *upsampling* e *downsampling*;
- aplicação de *BatchNorm* no gerador e no discriminador (exceto na primeira e última camadas);
- utilização da função de ativação *Leaky ReLU* no discriminador;
- uso do otimizador *Adam*, porém com coeficiente de momento menor que o padrão.

A maioria dos modelos de aprendizado profundo é mais robusta a essas escolhas.

Dificuldades no treinamento

Perda não informativa

- O gerador busca **minimizar** uma função de perda, dessa forma é tentador supor que:

↓ perda do gerador $\Rightarrow$ ↑ qualidade dos dados gerados

Contudo, essa relação não se sustenta nas GANs.

- O gerador é avaliado contra o **discriminador atual**, que também está em constante aprimoramento, de forma que os valores das funções de perda medidos em iterações diferentes do treinamento **não são comparáveis entre si**.
 - Ex.: A perda do gerador pode inclusive **aumentar** ao longo do tempo, mesmo enquanto a qualidade visual das imagens melhora nitidamente.
- Essa ausência de correlação entre perda do gerador e qualidade dos dados gerados dificulta o **monitoramento** do treinamento de GANs.

Dificuldades no treinamento

Mode dropping e Mode collapse

Mode dropping (“descarte de modo”)

- O gerador produz amostras sintéticas plausíveis, mas que representam apenas um subconjunto dos dados reais.

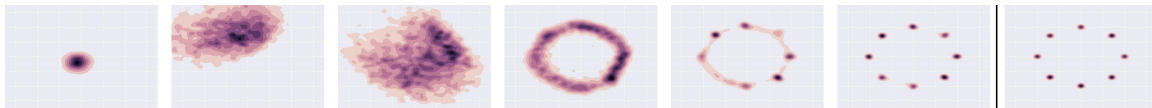
Mode collapse (“colapso de modo”)

- Caso extremo de mode dropping
- O gerador ignora (total ou parcialmente) a variável latente z e colapsa as amostras sintéticas em um ou poucos pontos.

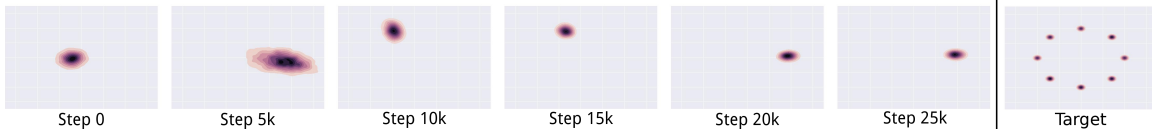
Dificuldades no treinamento

Mode dropping e Mode collapse

Comportamento desejado



Mode collapse (“colapso de modo”)

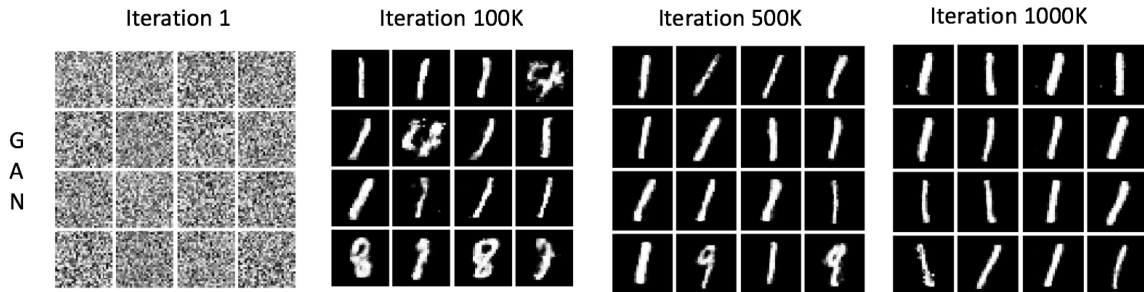


Adaptado de (L. Metz et al., 2017).

Dificuldades no treinamento

Mode dropping e Mode collapse

■ **Mode collapse** (“colapso de modo”)



Adaptado de (L. Mi *et al.*, 2018).

Dificuldades no treinamento

Desvanecimento dos gradientes

- Discriminador excessivamente forte (domina) em relação ao gerador.
 - Quando o discriminador distingue perfeitamente os dados reais dos sintéticos, ele não fornece mais nenhum sinal de aprendizado útil para o gerador (gradientes enviados ao gerador tornam-se muito pequenos), tornando o aprendizado lento ou paralisando-o.
- Esse fenômeno ocorre especialmente quando as distribuições de dados reais e sintéticos não possuem sobreposição.
 - As distribuições reais e sintéticas podem realmente ser disjuntas; os dados podem estar concentrados em variedades [*manifolds*] de baixa dimensão com pouca ou nenhuma sobreposição e, nesse caso, os gradientes são muito pequenos.

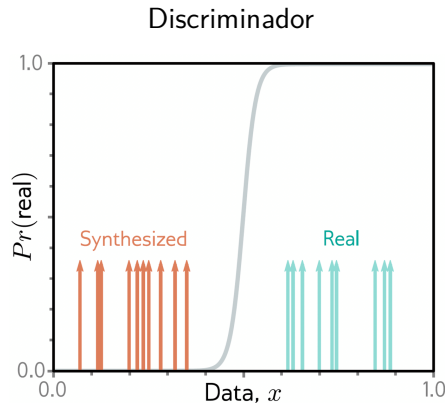
Dificuldades no treinamento

Desvanecimento dos gradientes

Quando a distribuição induzida pelo gerador $p_g(\mathbf{x})$ é muito diferente da distribuição verdadeira dos dados $p_{\text{dados}}(\mathbf{x})$, o discriminador tipicamente classifica as amostras sintéticas com alta confiança. Nesse caso,

$$D_\phi(G_\theta(\mathbf{z})) \approx 0,$$

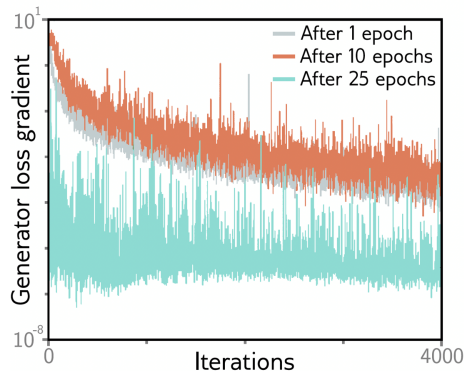
e o sinal de aprendizado propagado ao gerador torna-se muito pequeno.



Dificuldades no treinamento

Desvanecimento dos gradientes: evidência empírica

- Se o gerador da DCGAN é congelado (após certo número de épocas) e o discriminador continua a ser treinado:
 - A performance do discriminador melhora.
 - Os gradientes do gerador diminuem drasticamente.
- É necessário balancear cuidadosamente o gerador e o discriminador.



S. J. D. Prince. *Understanding Deep Learning*, The MIT Press, 2023.

Dificuldades no treinamento

Desvanecimento dos gradientes: perda não saturante

- **No início do treinamento**, as amostras sintéticas são bem diferentes das reais e o discriminador as identifica com facilidade: o termo $\log(1 - D_\phi(G_\theta(z)))$ satura.
- Na prática, para mitigar o desvanecimento dos gradientes, utiliza-se a perda não saturante [*non-saturating loss*] para o gerador:

$$\begin{aligned}\mathcal{L}_G^{\text{NS}} &= -\mathbb{E}_{\tilde{x} \sim p_g} [\log D_\phi(\tilde{x})] \\ &= -\mathbb{E}_{z \sim p_z} [\log D_\phi(G_\theta(z))], \quad \tilde{x} = G_\theta(z) \\ &\approx -\frac{1}{m} \sum_{j=1}^m \log D_\phi(G_\theta(z_j)), \quad z_j \sim p_z\end{aligned}$$

Dificuldades no treinamento

Desvanecimento dos gradientes: perda não saturante

Trocar o rótulo $\Rightarrow$ trocar a perda

As duas perdas do gerador se originam da **mesma** entropia cruzada binária [ECB]; o que muda é o rótulo que atribuímos às amostras sintéticas:

	rótulo nas sintéticas	operação	perda do gerador
minimax original	0 (verdadeiro)	maximizar a ECB	$\mathcal{L}_G^{\text{MM}} = \mathbb{E}_{z \sim p_z} [\log(1 - D_\phi(G_\theta(z)))]$
não saturante	1 (trocado)	minimizar a ECB	$\mathcal{L}_G^{\text{NS}} = -\mathbb{E}_{z \sim p_z} [\log D_\phi(G_\theta(z))]$

Na implementação chama-se a **mesma** função de entropia cruzada binária nos dois casos: para obter a perda não saturante basta passar 1 **como alvo** das amostras sintéticas ao treinar o gerador.

Dificuldades no treinamento

Desvanecimento dos gradientes: perda não saturante

No algoritmo de treinamento, a atualização do gerador passa a ser

$$\theta \leftarrow \theta - \eta_G \nabla_{\theta} \left\{ -\frac{1}{m} \sum_{j=1}^m \log D_{\phi}(G_{\theta}(z_j)) \right\}$$

O restante permanece idêntico: a amostragem, os k passos do discriminador e a atualização de ϕ não mudam.

Observação

Com a perda não saturante **o jogo deixa de ser de soma zero** e não pode mais ser descrito por uma única função de valor.

Dificuldades no treinamento

Desvanecimento dos gradientes: perda não saturante vs. perda minimax

- Perda não saturante ($\mathcal{L}_G^{\text{NS}}$):

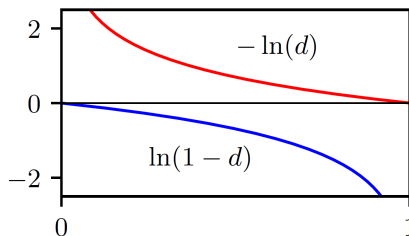
- Maximiza a probabilidade dos dados sintéticos serem classificados como reais.
- Fornece um sinal de gradiente com magnitude maior no início do treinamento (aprendizado mais rápido).

- Perda minimax ($\mathcal{L}_G^{\text{MM}}$):

- Minimiza a probabilidade dos dados sintéticos serem classificados como sintéticos.
- Fornece um sinal de gradiente com magnitude pequena no início do treinamento (desvanecimento dos gradientes).

No início do treinamento $d = D_\phi(G_\theta(z)) \approx 0$:

- $\mathcal{L}_G^{\text{NS}}$: a curva de $-\log d$ é **íngreme**;
- $\mathcal{L}_G^{\text{MM}}$: a curva de $\log(1 - d)$ é **muito menos íngreme**.



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Dificuldades no treinamento

Desvanecimento dos gradientes: perda minimax e a divergência de Jensen–Shannon

Análise da Função de Perda

- **No caso idealizado do jogo minimax original, com discriminador ótimo**, a função de perda *minimax* pode ser reescrita e interpretada como a divergência de Jensen–Shannon entre a distribuição dos dados p_{dados} e a induzida pelo gerador p_g .
- Para um gerador fixo G_θ , o discriminador ótimo D_ϕ^* é dado por:

$$D_\phi^*(\mathbf{x}) = \frac{p_{\text{dados}}(\mathbf{x})}{p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x})}$$

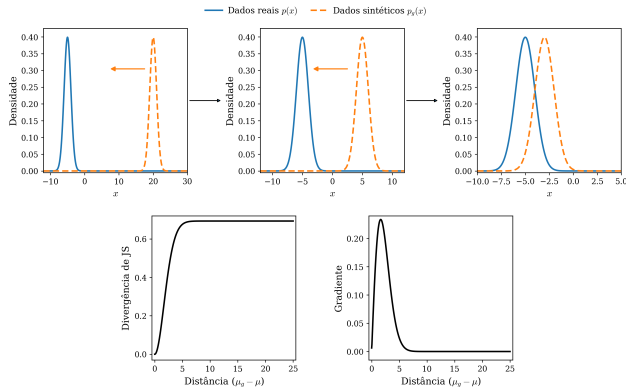
Dificuldades no treinamento

Desvanecimento dos gradientes: perda minimax e a divergência de Jensen–Shannon

$$\begin{aligned}
 -\mathcal{L}_{D^*} &= \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{dados}}} [\log D_{\phi}^*] + \frac{1}{2} \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D_{\phi}^*(G_{\theta}(\mathbf{z})))] \\
 &= \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{dados}}} [\log D_{\phi}^*(\mathbf{x})] + \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_g} [\log(1 - D_{\phi}^*(\mathbf{x}))] \\
 &= \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{dados}}} \left[\log \frac{p_{\text{dados}}(\mathbf{x})}{p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \left(1 - \frac{p_{\text{dados}}(\mathbf{x})}{p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x})} \right) \right] \\
 &= \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{dados}}} \left[\log \frac{p_{\text{dados}}(\mathbf{x})}{p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_g(\mathbf{x})}{p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x})} \right] \\
 &= \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{dados}}} \left[\log \frac{p_{\text{dados}}(\mathbf{x})}{2 \cdot \frac{1}{2} (p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x}))} \right] + \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_g(\mathbf{x})}{2 \cdot \frac{1}{2} (p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x}))} \right] \\
 &= \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{dados}}} \left[\log \frac{p_{\text{dados}}(\mathbf{x})}{\frac{1}{2} (p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x}))} - \log 2 \right] + \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_g(\mathbf{x})}{\frac{1}{2} (p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x}))} - \log 2 \right] \\
 &= \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{dados}}} \left[\log \frac{p_{\text{dados}}(\mathbf{x})}{\frac{1}{2} (p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x}))} \right] + \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_g(\mathbf{x})}{\frac{1}{2} (p_{\text{dados}}(\mathbf{x}) + p_g(\mathbf{x}))} \right] - \log 2 \\
 &= \frac{1}{2} D_{KL} \left(p_{\text{dados}} \parallel \frac{p_{\text{dados}} + p_g}{2} \right) + \frac{1}{2} D_{KL} \left(p_g \parallel \frac{p_{\text{dados}} + p_g}{2} \right) - \log 2 \\
 &= \boxed{D_{JS}(p_{\text{dados}} \parallel p_g) - \log 2}
 \end{aligned}$$

Dificuldades no treinamento

Desvanecimento dos gradientes: perda minimax e a divergência de Jensen–Shannon



Adaptado de Z. Wang et al., "Generative Adversarial Networks in Computer Vision: A Survey and Taxonomy," *ACM Comput. Surv.*, vol. 54, no. 2, Art. no. 37, Mar. 2022.

Dificuldades no treinamento

Desvanecimento dos gradientes: perda minimax e a divergência de Jensen–Shannon

Desprezando a constante aditiva, no jogo minimax idealizado, considerando o discriminador ótimo para cada configuração do gerador, o problema de otimização do gerador se torna:

$$\hat{\theta} = \arg \min_{\theta} [-\mathcal{L}_{D^*}] = \arg \min_{\theta} [D_{JS}(p_{\text{dados}} \parallel p_g)]$$

Os dois termos da divergência de **Jensen–Shannon** podem ser interpretados como **cobertura** e **qualidade**:

$$\begin{aligned} D_{JS}(p_{\text{dados}} \parallel p_g) &= \frac{1}{2} D_{KL} \left(p_{\text{dados}} \parallel \frac{p_{\text{dados}} + p_g}{2} \right) + \frac{1}{2} D_{KL} \left(p_g \parallel \frac{p_{\text{dados}} + p_g}{2} \right) \\ &= \underbrace{\frac{1}{2} \int p_{\text{dados}}(x) \log \frac{2 p_{\text{dados}}(x)}{p_{\text{dados}}(x) + p_g(x)} dx}_{\text{Cobertura}} + \underbrace{\frac{1}{2} \int p_g(x) \log \frac{2 p_g(x)}{p_{\text{dados}}(x) + p_g(x)} dx}_{\text{Qualidade}}. \end{aligned}$$

- **Cobertura**: penaliza regiões onde há dados reais mas o gerador não produz amostras.
- **Qualidade**: penaliza regiões onde o gerador produz amostras mas não há dados reais.

Dificuldades no treinamento

Desvanecimento dos gradientes: perda minimax e a divergência de Jensen–Shannon

Embora para o discriminador ótimo D_ϕ^* tenhamos termos associados à **cobertura** e à **qualidade**,

$$-\mathcal{L}_{D^*} = \underbrace{\frac{1}{2} D_{KL} \left(p_{\text{dados}} \parallel \frac{p_{\text{dados}} + p_g}{2} \right)}_{\text{Cobertura}} + \underbrace{\frac{1}{2} D_{KL} \left(p_g \parallel \frac{p_{\text{dados}} + p_g}{2} \right)}_{\text{Qualidade}} - \log 2,$$

o termo associado à **cobertura** não contribui diretamente para a atualização do gerador. Para ver isso, voltamos à forma **antes** de substituir D_ϕ^* , isto é, considerando um discriminador **fixo** D_ϕ no passo de atualização do gerador:

$$-\mathcal{L}_D = \underbrace{\frac{1}{2} \mathbb{E}_{x \sim p_{\text{dados}}} [\log D_\phi(x)]}_{\text{não depende de } \theta \text{ para } D_\phi \text{ fixo}} + \frac{1}{2} \mathbb{E}_{z \sim p_z} [\log (1 - D_\phi(G_\theta(z)))].$$

A parcela dos dados reais, associada à **cobertura**, não depende de θ quando ϕ está fixo e, portanto, é constante em relação aos parâmetros do gerador. Assim, ela não contribui para o gradiente utilizado em sua atualização. Esta é a razão putativa para o *mode dropping*.

Referências

Referências principais

 Simon J. D. Prince.
Understanding Deep Learning.
The MIT Press, 2023.
[Capítulo 15]
Disponível em: <https://udlbook.github.io/udlbook/>

 Christopher M. Bishop e Hugh Bishop.
Deep Learning: Foundations and Concepts.
Springer, 2024.
[Capítulo 17]
Disponível em: <https://www.bishopbook.com/>

Referência complementar

 David Foster.
Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play.
2ª ed. O'Reilly Media, 2023.
[Capítulo 4]

Slides construídos com base nos livros supracitados, os quais estão disponibilizados publicamente pelos seus respectivos autores.